

Анализ графов

Пентегова Мария Сергеевна

Группа: 25.Б41-мм

Кафедра: системного программирования

Научный руководитель: Григорьев Семён Вячеславович

Номер семестра практики: 2

1. Введение

GraphBLAS позволяет формулировать алгоритмы анализа графов в терминах операций линейной алгебры над разреженными матрицами и векторами. Цель данной работы — реализовать обход графа в ширину (BFS) в классическом и матрично-алгебраическом вариантах и исследовать зависимость их производительности от структуры входного графа.

2. Постановка задачи и реализация

Реализованы четыре варианта BFS: классический BFS с построением массива родителей (parent-BFS), классический multisource BFS, GraphBLAS BFS с вычислением уровней и GraphBLAS multisource BFS. GraphBLAS-варианты реализованы с использованием SuiteSparse:GraphBLAS.

В классическом варианте граф хранится в формате CSR (Compressed Sparse Row). Для вершины v её соседи находятся в диапазоне

$$[\text{row_ptr}[v], \text{row_ptr}[v + 1])$$

массива `col_idx`. BFS использует очередь, а при первом посещении вершины u записывается $\text{parent}[u] = v$. В multisource-варианте очередь изначально содержит несколько исходных вершин. Сложность классического BFS составляет $O(|V| + |E|)$.

Классическая реализация написана непосредственно на CSR, а не взята из сторонней библиотеки. Это позволяет сформировать простой и воспроизводимый контрольный вариант эксперимента: тот же исходный CSR используется непосредственно классическим BFS и для построения матрицы GraphBLAS. Таким образом, оба варианта работают с одним набором вершин и рёбер, а различие в способе выполнения обхода задаётся постановкой эксперимента.

Готовые реализации BFS из BGL и LAGraph не используются как измеряемые реализации в данном эксперименте. Они являются существующими реализациями алгоритмов и могут служить ориентиром при реализации алгоритма, однако целью работы не является сравнение библиотек. Непосредственное включение этих реализаций в измерения изменило бы объект эксперимента: помимо различия между классическим и матрично-алгебраическим способами выполнения BFS пришлось бы учитывать особенности конкретных библиотечных реализаций, их структур данных и оптимизаций. Поэтому классический вариант намеренно реализован самостоятельно на CSR как простой и прозрачный контрольный вариант.

В GraphBLAS граф представляется булевой матрицей смежности A , а текущий фронт — разреженным булевым вектором f_k . Следующий фронт вычисляется матрично-векторной операцией

$$f_{k+1} = f_k A$$

на булевом semiring LOR-LAND. Уже посещённые вершины исключаются с помощью маски. Номер итерации определяет уровень вершины в BFS. В multisource-варианте начальный вектор содержит несколько источников. Таким образом, классическая реализация представляет собой контрольный вариант последовательного BFS, а GraphBLAS-реализация выражает тот же обход через операции над разреженными матрицами и векторами. Такой подход соответствует назначению GraphBLAS [1].

Программа написана на C11. Для измерения времени используется `clock_gettime(CLOCK_MONOTONIC)`. Перед серией измерений GraphBLAS-вариантов выполняется прогревочный запуск. Каждый вариант запускается 10 раз; для каждой серии вычисляются среднее, минимальное и максимальное время.

Матрица GraphBLAS импортируется из CSR один раз до начала измерений. Время её построения выводится отдельно и не включается во время BFS. Таким образом, в измерение BFS входит полное выполнение соответствующей GraphBLAS-функции на уже подготовленном представлении графа, а стоимость однократного построения матрицы учитывается отдельно.

3. Эксперимент

Для экспериментов выбраны семь графов из SuiteSparse Matrix Collection, содержащей набор разреженных матриц для тестирования алгоритмов работы с разреженными данными [4].

Для однократного BFS источником выбирается вершина максимальной степени. Такой выбор обеспечивает воспроизводимое определение источника для каждого графа. Для multisource BFS используются четыре источника с индексами 0, $n/4$, $n/2$ и $3n/4$.

Граф	$ V $	$ E $	Тип	$\deg(s)$
com-Orkut	3.07M	117M	социальный	33313
soc-LiveJournal1	4.85M	69M	социальный	38660
road_usa	23.95M	28.85M	дорожный	4
roadNet-CA	1.97M	2.77M	дорожный	3
web-Google	0.92M	5.10M	веб-граф	6332
cit-Patents	3.77M	16.52M	цитирование	852
com-Amazon	0.33M	0.93M	социальный	546

Таблица 1: Характеристики исследуемых графов.

Перед измерениями для GraphBLAS-вариантов выполняется один прогревочный запуск. Затем каждый из четырёх вариантов BFS запускается 10 раз. Время измеряется в микросекундах; для каждой реализации вычисляются среднее, минимальное и максимальное значения.

Граф	CP	CM	GB-L	GB-M
com-Orkut	2652628	2700768	486500	420307
soc-LiveJournal1	1915627	1976468	358255	331458
road_usa	1451387	1588466	25839879	22121848
roadNet-CA	127543	110558	663630	670218
web-Google	100233	80683	101149	106972
cit-Patents	17500	4730	181639	46757
com-Amazon	38967	31974	61923	62144

Таблица 2: Среднее время BFS по 10 запускам, мкс. CP — Classic Parent, CM — Classic Multisource, GB-L — GraphBLAS Level, GB-M — GraphBLAS Multisource.

4. Анализ результатов

На com-Orkut и soc-LiveJournal1 GraphBLAS Level показал меньшее среднее время: 486 500 и 358 255 мкс против 2 652 628 и 1 915 627 мкс для CP. Отношение времени CP к GB-L составляет примерно 5.45 и 5.35 соответственно.

На road_usa и roadNet-CA, напротив, GraphBLAS Level занял 25 839 879 и 663 630 мкс против 1 451 387 и 127 543 мкс для CP. На web-Google времена близки: 100 233 и 101 149 мкс. На cit-Patents и com-Amazon классический вариант также оказался быстрее.

Таким образом, в исследованной выборке производительность зависит от структуры графа. Одним из возможных факторов является ширина BFS-фронтов: широкие фронты позволяют выполнить большой объём работы матричной операцией, тогда как при узких фронтах накладные расходы GraphBLAS могут становиться существеннее. В текущем эксперименте размеры фронтов отдельно не измерялись, поэтому это объяснение является гипотезой для дальнейшей проверки.

Полученные результаты также показывают, что само использование матрично-алгебраического представления не гарантирует уменьшения времени на каждом типе графа. Эффект зависит от того, насколько структура конкретного графа и возникающие в BFS фронты подходят для обработки разреженными матричными операциями.

5. Тестирование

Корректность реализации проверяется с помощью CTest и assert.h. Тесты выполняются на небольших графах и проверяют посещение вершин, вычисление уровней, построение родительского дерева, multisource BFS и соответствие результатов классической и GraphBLAS-реализаций.

Классический parent-BFS используется как контрольный вариант: для одних и тех же источников сравниваются достижимость вершин и соответствующие расстояния. Все тесты проходят успешно: 100% passed.

6. Заключение

В работе реализованы четыре варианта BFS на C: два классических варианта на CSR и два варианта на SuiteSparse:GraphBLAS. Классическая реализация выполнена самостоятельно как простой последовательный контрольный вариант на CSR. Это позволяет сравнивать классический и матрично-алгебраический способы выполнения BFS на одном исходном графе.

Готовые реализации BGL и LAGraph не используются как измеряемые baseline, поскольку целью работы является исследование реализации BFS через GraphBLAS API, а не сравнение библиотек. Самостоятельный классический BFS используется как контрольный алгоритмический вариант. Стоимость однократного построения GraphBLAS-представления измеряется отдельно.

Эксперименты проведены на семи графах из SuiteSparse Matrix Collection. Для каждого графа выполнены прогревочный запуск и десять измерений каждого варианта. Построение GraphBLAS-матрицы выполняется один раз и измеряется отдельно.

Результаты показывают зависимость производительности от структуры графа. На com-Orkut и soc-LiveJournal11 GraphBLAS Level выполнялся примерно в 5.45 и 5.35 раза быстрее CP, тогда как на road_usa, roadNet-CA, cit-Patents и com-Amazon меньшее время наблюдалось у классического варианта. На web-Google результаты были близки.

Для дальнейшего эксперимента целесообразно измерять размер фронта на каждом уровне BFS, число обработанных рёбер и стоимость отдельных GraphBLAS-операций. Это позволит проверить связь структуры фронтов с наблюдаемой разницей производительности.

Репозиторий проекта:

https://github.com/MariaPentegova/graph_blas_bfs-2

Список литературы

- [1] GraphBLAS Forum. *The GraphBLAS C API Specification. Version 2.1*, 2023.
https://graphblas.org/docs/GraphBLAS_API_C_v2.1.0.pdf
- [2] Davis, T. A. *SuiteSparse:GraphBLAS*. Официальный репозиторий проекта.
<https://github.com/DrTimothyAldenDavis/GraphBLAS>
- [3] Davis, T. A. *SuiteSparse*. Официальный репозиторий проекта.
<https://github.com/DrTimothyAldenDavis/SuiteSparse>
- [4] Davis, T. A., Hu, Y. *The University of Florida Sparse Matrix Collection*. ACM Transactions on Mathematical Software, 38(1), 2011.
<https://sparse.tamu.edu/>
- [5] Kitware. *CTest Documentation*.
<https://cmake.org/cmake/help/latest/manual/ctest.1.html>